



Τμήμα Εφαρμοσμένης Πληροφορικής
Εργασία στο μάθημα Γλώσσες Προγραμματισμού και Μεταγλωττιστές

“Σημασιολογική Ανάλυση και Παραγωγή Κώδικα”

Τσιμπίνη Πετρούλα ics23085

Ιούνιος 2026

1.Εισαγωγή

Η παρούσα εργασία αφορά την υλοποίηση ενός μεταγλωττιστή για τη γλώσσα περιγραφής αριθμητικών εκφράσεων σε προθεματική μορφή. Ο μεταγλωττιστής δέχεται τον πηγαίο κώδικα της γλώσσας και παράγει κώδικα JVM assembly, έτοιμο για μεταγλώττιση σε εκτελέσιμο αρχείο.

Η υλοποίηση χρησιμοποιεί τα εργαλεία Flex (λεκτική ανάλυση) και Bison (συντακτική/σημασιολογική ανάλυση) πάνω στον σκελετό (jvmLExp.l, jvmLExp.y, symbolTable, codeFacilities, jvmLangTypesFunctions) που μας δώσατε στο eclass.

Τα δύο αρχεία που τροποποιήθηκαν είναι το jvmLExp.l που είναι ο λεκτικός αναλυτής (Flex) και jvmLExp.y που είναι ο συντακτικός/σημασιολογικός αναλυτής (Bison).

2.Λεκτική Ανάλυση

Στον δοσμένο σκελετό υπήρχαν μόνο τα tokens T_start και T_end. Προστέθηκαν όλα τα υπόλοιπα tokens που απαιτούνται από τη γλώσσα: η λέξη print, οι δηλώσεις τύπου int/float, ο τελεστής αύξησης ++, οι αριθμητικοί τελεστές, οι συγκριτικοί τελεστές, ο τελεστής ανάθεσης =, τα σύμβολα της έκφρασης υπό συνθήκη ? και : και οι παρενθέσεις ().

Για τις σταθερές χρησιμοποιήθηκαν δύο κανονικοί ορισμοί: INT για ακέραιες σταθερές και FLOAT για πραγματικές, με την τιμή του lexeme να αποθηκεύεται ως string στο yyval.lexical. Οι μεταβλητές αναγνωρίζονται από τον κανόνα VARIABLE, όπως δίνεται στον σκελετό. Μία σημαντική προσθήκη είναι ο global μετρητής g_paren_depth, που χρησιμοποιείται από τον μηχανισμό ανάκαμψης σφαλμάτων.

Ορίσαμε τις κανονικές εκφράσεις:

DIGITS	[0-9]	- ψηφία 0-9
LETTER	[a-zA-Z]	- γράμματα
VARIABLE	{LETTER}{(LETTER){DIGITS} _)*	- αναγνωριστικά μεταβλητών
FLOAT	{DIGITS}+\.{DIGITS}+	- πραγματικοί αριθμοί
INT	{DIGITS}+	- ακέραιοι αριθμοί
newline	\n \x0A \x0D\x0A	- νέα γραμμή

Αναγνώριση των tokens:

Δεσμευμένες λέξεις: start, end, print, int, float που επιστρέφουν τα αντίστοιχα tokens.

Τελεστές: ++, +, -, *, /, ==, >, <, =, ?, : που επιστρέφουν τα αντίστοιχα tokens.

Παρενθέσεις: Κάθε (αυξάνει τον μετρητή g_paren_depth και κάθε) τον μειώνει. Ο μετρητής αυτός χρησιμοποιείται για την ανάκαμψη από σφάλματα, ώστε ο αναλυτής να ξέρει που τελειώνει μια εντολή.

Αριθμοί: Οι πραγματικοί (FLOAT) και ακέραιοι (INT) αριθμοί αποθηκεύουν την τιμή τους με strdup(yytext) στο yyval.lexical.

Αναγνωριστικά: Τα ονόματα μεταβλητών αποθηκεύονται επίσης στο yyval.lexical.

Νέα γραμμή: Αυξάνει τον μετρητή code_line για σωστή αναφορά γραμμής στα μηνύματα σφάλματος.

Κενά: Αγνοούνται

Άγνωστοι χαρακτήρες: Εκτυπώνεται μήνυμα λεκτικού σφάλματος με την γραμμή που εντοπίστηκε.

Κώδικας:

```
/*
Grammar with semantics and type checking.
Lexical Analyser - jvmLExp.l
*/

%{
#include <stdlib.h>
#include <string.h>
int code_line = 1;
int g_paren_depth = 0; /* tracks '(' / ')' nesting, used for error recovery */
}%

DIGITS    [0-9]
LETTER    [a-zA-Z]
VARIABLE  {(LETTER){(LETTER)|(DIGITS)|_} *}
FLOAT     {(DIGITS)+\.{(DIGITS)}+
INT       {(DIGITS)}+
newline   \n|\x0A|\x0D\x0A

%%

"start"   {return T_start;}
"end"     { return T_end; }
"print"   { return T_print;}
"int"     { return T_int; }
"float"   { return T_float;}
"++"      { return T_inc;}
"+"       { return T_plus; }
"-"       { return T_minus;}
"*"       { return T_mul; }
"/"       { return T_div;}
"=="      { return T_eq;}
">"       { return T_gt;}
"<"       { return T_lt; }
"="       { return T_assign;}
"?"       { return T_qmark; }
":"       { return T_colon; }
"("       { g_paren_depth++; return T_lparen; }
")"       { g_paren_depth--; return T_rparen; }

{FLOAT}   { yyval.lexical = strdup(yytext); return T_fnum;}
{INT}     { yyval.lexical = strdup(yytext); return T_num;}

{VARIABLE} { yyval.lexical = strdup(yytext); return T_id; }

{newline} { code_line++;}
[ \t]     { /* nothing */ }
.         {
    printf("Lexical Analysis: Unexpected String! :: (%s) . in line %d. \n", yytext, code_line);
}
Σ
%%
```

3.Συντακτικός Αναλυτής

Ο συντακτικός αναλυτής υλοποιεί τη γραμματική της γλώσσας σε προθεματική μορφή και ενσωματώνει τις σημασιολογικές ενέργειες που παράγουν τον κώδικα Jasmin.

3.1 Δήλωση των tokens, των τύπων και του %union

Αρχικά δηλώθηκαν όλα τα τερματικά σύμβολα. Με το %union ορίζεται ο τύπος των σημασιολογικών τιμών: τα T_id, T_num, T_fnum φέρουν λεκτική τιμή, η μη-τερματική expr φέρει εγγραφή se (τύπος και place) και το relop φέρει τον δείκτη της σχέσης. Ο τύπος se.type είναι σημαντικός καθώς διαδίδει τον τύπο (int ή real) κάθε υποέκφρασης προς τα πάνω, ώστε να γίνονται οι κατάλληλες μετατροπές.

```
%union{
    int    intval;
    char   *lexical;
    struct {
        ParType type;
        char   *place;
    } se;
    RelationType relopIndex;
}

%token T_start "start"
%token T_end   "end"
%token T_print "print"
%token T_int   "int"
%token T_float "float"
%token T_inc   "++"
%token T_plus  "+"
%token T_minus "-"
%token T_mul   "*"
%token T_div   "/"
%token T_eq    "=="
%token T_gt    ">"
%token T_lt    "<"
%token T_assign "="
%token T_qmark "?"
%token T_colon ":"
%token T_lparen "("
%token T_rparen ")"

%token <lexical> T_id
%token <lexical> T_num
%token <lexical> T_fnum

%type <se>      expr
%type <relopIndex> relop

%%
```

3.2 Δομή του προγράμματος και εντολές

Ο κανόνας program εκπέμπει το preamble της κλάσης (όνομα κλάσης = όνομα προγράμματος) αμέσως μετά το όνομα, αρχικοποιεί τον πίνακα συμβόλων και στο τέλος εμφανίζει return και end method.

```
program:
    T_start T_id
    {
        create_preamble($2);
        symbolTable= NULL;
    }
    stmts T_end
    {
        insertINSTRUCTION("return");
        insertINSTRUCTION(".end method\n");
    }
    ;
```

Οι εντολές stmt χρησιμοποιούνται για την δήλωση int/float, την ανάθεση και την εκτύπωση καθώς δηλώνει και έναν κανόνα ανάκαμψης σφαλμάτων. Στη δήλωση καλείται η addvar σε περίπτωση που η μεταβλητή υπάρχει ήδη, παράγεται σφάλμα διπλής δήλωσης.

```
stmt:
    /*variable declaration: (int x) */
    T_lparen T_int T_id T_rparen
    {
        if (!addvar(&symbolTable, $3, type_integer))
            ERR_VAR_DECL($3, code_line);
    }

    /*variable declaration:(float y) */
    | T_lparen T_float T_id T_rparen
    {
        if (!addvar(&symbolTable, $3, type_real))
            ERR_VAR_DECL($3, code_line);
    }
```

Εντολή ανάθεσης: (= var expr)

Ο έλεγχος αν η μεταβλητή έχει δηλωθεί γίνεται πριν αναλυθεί η expr. Έτσι, αν η μεταβλητή δεν έχει δηλωθεί, το YYERROR ενεργοποιείται ενώ στη στοίβα του parser βρίσκονται μόνο τα "(= id" και ο κανόνας ανάκαμψης μπορεί να παραλείψει ολόκληρη την εσφαλμένη εντολή. Μετά την ανάλυση της expr, εφαρμόζεται μετατροπή τύπου αν χρειάζεται (f2i ή i2f) και εκπέμπεται η κατάλληλη εντολή store.

```

| T_lparen T_assign T_id
{
    if (!lookup(symbolTable,$3))
        ERR_VAR_MISSING($3,code_line);
}
expr T_rparen
{
    ParType vartype= lookup_type(symbolTable, $3);
    int varpos= lookup_position(symbolTable, $3);
    if (vartype== type_integer && $5.type == type_real)
        insertINSTRUCTION("f2i");
    else if (vartype== type_real && $5.type == type_integer)
        insertINSTRUCTION("i2f");
    insertSTORE(vartype, varpos);
}

```

Εκτύπωση:(print expr)

Εκπέμπεται το πρότυπο swap και invokevirtual println, επιλέγοντας την μέθοδο println(I) ή println(F) ανάλογα με τον τύπο της έκφρασης.

```

/* Print (print expr) */
| T_lparen T_print expr T_rparen
{
    insertINSTRUCTION("getstatic java/lang/System/out Ljava/io/PrintStream;");
    insertINSTRUCTION("swap");
    if ($3.type== type_integer)

insertINVOKEVIRTUAL("java/io/PrintStream/println",type_integer,type_void);
    else
        insertINVOKEVIRTUAL("java/io/PrintStream/println",type_real, type_void);
}

/*

```

3.3 Αριθμητικές εκφράσεις και μετατροπές τύπου

Η συνάρτηση coerce_binary είναι αυτή που εκτελεί την αυτόματη μετατροπή τύπου. Σύμφωνα με τη διάταξη στη στοίβα [αριστερό][δεξί] (το δεξί στην κορυφή):

- int + int :αποτέλεσμα int, καμία μετατροπή.
- real + real: αποτέλεσμα real, καμία μετατροπή.
- real + int :ο ακέραιος (δεξί) είναι στην κορυφή άρα αρκεί i2f.
- int + real: ο ακέραιος (αριστερό) είναι «δεύτερος» στη στοίβα, άρα swap, i2f, swap.


```

static ParType coerce_binary(ParType left, ParType right)
{
    if (left== type_integer && right== type_integer) return type_integer;
    if (left ==type_real && right==type_real)    return type_real;
    if (left ==type_real && right== type_integer) {
        insertINSTRUCTION("i2f");
    } else {
        insertINSTRUCTION("swap");
        insertINSTRUCTION("i2f");
        insertINSTRUCTION("swap");
    }
    return type_real;
}

```

Οι κανόνες +, -, *, / καλούν την `coerce_binary` για τους δύο τελεστέους τους και έπειτα την `insertOPERATION` με την κατάλληλη πράξη (add/sub/mul/div). Το πρόθεμα (i ή f) επιλέγεται αυτόματα από τον τύπο που επιστρέφει η `coerce_binary`, παράγοντας `iadd/fadd`, `imul/fmul` κλπ.

```

/*Addition:+ expr expr */
| T_plus expr expr
{
    ParType rt= coerce_binary($2.type, $3.type);
    insertOPERATION(rt, "add");
    $$ .type =rt;
}

/*subtraction:- expr expr */
| T_minus expr expr
{
    ParType rt= coerce_binary($2.type, $3.type);
    insertOPERATION(rt, "sub");
    $$ .type =rt;
}

/*multiplication:* expr expr */
| T_mul expr expr
{
    ParType rt= coerce_binary($2.type, $3.type);
    insertOPERATION(rt, "mul");
    $$ .type= rt;
}

/*division:/ expr expr */
| T_div expr expr
{
    ParType rt= coerce_binary($2.type, $3.type);
    insertOPERATION(rt, "div");
    $$ .type =rt;
}

```


3.4 Φόρτωση των τιμών και τελεστής ++

Ακέραια και πραγματική σταθερά: ο ακέραιος μπαίνει στη στοίβα με την `pushInteger`, ο πραγματικός με την `"ldc"`. Κατά την φόρτωση της μεταβλητής γίνεται έλεγχος ύπαρξης, εύρεση τύπου/θέσης και εκπομπή `iload/fload`.

Για τον τελεστή `++` (μόνο ακέραιες μεταβλητές) έχουμε διαφορετική ακολουθία ανάλογα με τη θέση:

- **Επιθεματικός (`var++`):** πρώτα `iload` της παλιάς τιμής στη στοίβα και μετά `iinc` της μεταβλητής στη μνήμη.
- **Προθεματικός (`++var`):** πρώτα `iinc` και μετά `iload` της νέας τιμής.

```
/* Post-increment: var++
 * load old value onto stack then increase the variable in memory */
| T_id T_inc
{
    if (!lookup(symbolTable, $1))
        ERR_VAR_MISSING($1, code_line);
    if (lookup_type(symbolTable, $1) != type_integer)
        ERR_TYPE_MISMATCH("int", code_line);
    int vp = lookup_position(symbolTable, $1);
    insertLOAD(type_integer, vp);
    insertIINC(vp, 1);
    $$ .type = type_integer;
}

/* pre-increment: ++var
 * Increase the variable in memory first then load the new value */
| T_inc T_id
{
    if (!lookup(symbolTable, $2))
        ERR_VAR_MISSING($2, code_line);
    if (lookup_type(symbolTable, $2) != type_integer)
        ERR_TYPE_MISMATCH("int", code_line);
    int vp = lookup_position(symbolTable, $2);
    insertIINC(vp, 1);
    insertLOAD(type_integer, vp);
    $$ .type = type_integer;
}
```

3.5 Έκφραση υπό συνθήκη με άλματα (backpatching μέσω labels)

Η έκφραση `(relop lhs rhs ? true_val : false_val)` μεταγλωττίζεται με άλματα, χρησιμοποιώντας τρία labels (`L1`, `L2`, `L3`). Η λογική που ακολουθείται είναι η εξής:

- **M1** (μετά τα `relop lhs rhs`, οπότε οι δύο τελεστέοι είναι στη στοίβα): δέσμευση L1, L2, L3, εκπομπή `if_icmp<relop> L1` (άλμα στο τμήμα αληθές), `goto L2` (αλλιώς στο ψευδές) και τοποθέτηση του L1.
- **M2** (μετά την αποτίμηση του `true`): εκπομπή `goto L3` (παράκαμψη του ψευδούς κλάδου) και τοποθέτηση του L2.
- **Τελική** (μετά την αποτίμηση του `false`): τοποθέτηση του L3. Ο τύπος της έκφρασης είναι ο τύπος του κλάδου.

```
| T_lparen relop expr expr
/* M1 */
{
    cond_l1= Label();
    cond_l2 =Label();
    cond_l3= Label();
    insertICMPOP($2,cond_l1);
    insertGOTO(cond_l2);
    insertLabel(cond_l1);
}
T_qmark expr
/* M2 */
{
    insertGOTO(cond_l3);
    insertLabel(cond_l2);
}
T_colon expr T_rparen
{
    insertLabel(cond_l3);
    $$ .type= $7.type;
}
```

3.6 Έκφραση μέσα σε παρενθέσεις και επίλυση συγκρούσεων

Ο κανόνας ``( expr )`` απλώς διαδίδει τη σημασιολογική τιμή της εσωτερικής έκφρασης. Η γραμματική εμφανίζει ένα `shift/reduce conflict` στην κατάσταση όπου, μετά από ένα `T_id`, ακολουθεί `++`: ο parser πρέπει να αποφασίσει αν θα κάνει `reduce` το `T_id` σε `expr` (φόρτωση μεταβλητής) ή `shift` το `++` (`post-increment`). Το Bison το επιλύει με την προεπιλογή `shift`, που είναι ακριβώς η επιθυμητή συμπεριφορά, το `++` συνδέεται με την προηγούμενη μεταβλητή. Πρόκειται για ένα καλό `conflict` που δεν επηρεάζει την ορθότητα.

```
/* Parenthesised expression:( expr ) */
| T_lparen expr T_rparen
{
    $$= $2;
}
;
```

```

/* Variable load */
| T_id
{
    if (!lookup(symbolTable, $1))
        ERR_VAR_MISSING($1, code_line);
    ParType vt= lookup_type(symbolTable, $1);
    int vp= lookup_position(symbolTable, $1);
    insertLOAD(vt, vp);
    $$ .type = vt;
}

/* Post-increment: var++
 * load old value onto stack then increase the variable in memory */
| T_id T_inc
{
    if (!lookup(symbolTable,$1))
        ERR_VAR_MISSING($1,code_line);
    if (lookup_type(symbolTable, $1)!= type_integer)
        ERR_TYPE_MISMATCH("int", code_line);
    int vp= lookup_position(symbolTable, $1);
    insertLOAD(type_integer, vp);
    insertIINC(vp, 1);
    $$ .type= type_integer;
}

```

3.7 Ανίχνευση και ανάκαμψη σφαλμάτων

Υλοποιήθηκαν τρία είδη ελέγχου, με αντίστοιχες εντολές που τυπώνουν περιγραφικό μήνυμα, καλούν την yyerror (αυξάνοντας τον μετρητή no_errors) και ενεργοποιούν YYERROR:

- **Διπλή δήλωση** (ERR_VAR_DECL): η μεταβλητή υπάρχει ήδη στον πίνακα συμβόλων.
- **Μη δηλωμένη μεταβλητή** (ERR_VAR_MISSING): χρήση μεταβλητής που δεν δηλώθηκε, σε ανάθεση ή σε έκφραση.
- **Ασυμβατότητα τύπου** (ERR_TYPE_MISMATCH): χρήση του ++ σε μη ακέραια μεταβλητή.

Για την ανάκαμψη χρησιμοποιείται ο ειδικός κανόνας (error. Όταν αποτύχει μια εντολή, το Bison κοιτάει τη στοίβα μέχρι το token error, τότε η συνάρτηση skip_to_stmt_end() καταναλώνει τα υπόλοιπα tokens της εσφαλμένης εντολής (χρησιμοποιώντας τον μετρητή g_paren_depth του lexer, ώστε να χειρίζεται σωστά και τις εμφωλευμένες παρενθέσεις) και η ανάλυση συνεχίζεται από την επόμενη εντολή. Έτσι αναφέρονται πολλαπλά σφάλματα σε ένα πέρασμα. Αν εντοπιστεί έστω και ένα σφάλμα, δεν παράγεται καθόλου κώδικας (No Code Generated).

```

void skip_to_stmt_end(void)
{
    /* g_paren_depth reflects how many '(' are still open,
       counting the outer statement paren plus any nested ones that
       were already consumed before the error occurred. We simply
       keep reading tokens (the lexer updates g_paren_depth itself)
       until we are back to the depth that existed before this
       statement's opening '(' was read for example depth 0. */
    int tok;
    while (g_paren_depth > 0){
        tok = yylex();
        if (tok == 0) break; /* EOF safety */
    }
}

int yyerror(const char * msg)
{
    fprintf(stderr, "ERROR: %s (line %d).\n", msg, code_line);
    no_errors++;
    return 0;
}

```

```

    if (no_errors != 0)
        fprintf(stderr, "No Code Generated.\n");
    print_symbol_table(symbolTable);
    return result;
}

```

4.Σχολιασμός Αποτελεσμάτων

Ο μεταγλωττιστής δοκιμάστηκε και στα εννέα παραδείγματα (simple1–simple9) καθώς και στο παράδειγμα σφαλμάτων. Σε κάθε περίπτωση ο παραγόμενος κώδικας Jasmin ταυτίζεται με τον αναμενόμενο της εκφώνησης. Παρακάτω σχολιάζεται κάθε παράδειγμα: η είσοδος, ο παραγόμενος κώδικας και η αντιστοίχιση με τους κανόνες της γραμματικής.

Παράδειγμα 1:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple1
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated ----

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
7
```

Σχολιασμός:

- Πηγαίος κώδικας: (print (+ 3 4)). Οι σταθερές 3 και 4 μπαίνουν στη στοίβα με iconst_3 / iconst_4 και ακολουθεί iadd, αφού και οι δύο τελεστές είναι int (η coerce_binary επιστρέφει int -> πρόθεμα i).
- Αλλαγές/προσθήκες: προστέθηκαν τα tokens T_print, T_plus, T_num, ο κανόνας έκφρασης για ακέραια σταθερά και για πρόσθεση (+ expr expr) καθώς και ο κανόνας (print expr) που εκπέμπει το πρότυπο getstatic / swap / invokevirtual println(I)V.
- Αποτέλεσμα εκτέλεσης: όπως φαίνεται στο screenshot, ο μεταγλωττιστής αναφέρει Errors found 0, ο jasmin παράγει το .class και η εκτέλεση σε Java τυπώνει 7, δηλαδή 3+4 που είναι το αναμενόμενο.

Παράδειγμα 2:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple2
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated ----

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
19
19.0
```

Σχολιασμός:

- Πηγαίος κώδικας: δύο εντολές print με τις (+ * 3 4 7) και (+ * 3.0 4.0 7.0). Η προθεματική (+ * 3 4 7) ισοδυναμεί με (3*4)+7: εκπέμπεται imul για τα 3,4 και μετά iadd με το 7 (το 7 ξεπερνά το όριο των iconst άρα μπαίνει με bipush 7).

- Αλλαγές/προσθήκες: προστέθηκε ο κανόνας πολλαπλασιασμού ($*$ expr expr) και η αναγνώριση πραγματικών σταθερών (T_fnum) που μπαίνουν στη στοίβα με ldc. Το πρόθεμα της πράξης (i ή f) επιλέγεται αυτόματα από τον τύπο που επιστρέφει η `coerce_binary`, παράγοντας `imul/iadd` για ακραίους και `fmul/fadd` για πραγματικούς.
- Αποτέλεσμα εκτέλεσης: το screenshot δείχνει δύο τιμές, 19 (ακέραια: $3*4+7$) και 19.0 (πραγματική: $3.0*4.0+7.0$) με την εκτύπωση να γίνεται με `println(I)V` και `println(F)V` αντίστοιχα.

Παράδειγμα 3:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple3
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated -----

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
3.3
8.5
```

Σχολιασμός:

- Πηγαίος κώδικας: `(print (+ 2.3 1))` και `(print (+ 4 4.5))` μικτές πράξεις οι οποίες δοκιμάζουν την αυτόματη μετατροπή τύπου.
- Αλλαγές/προσθήκες: η `coerce_binary` χειρίζεται τις δύο περιπτώσεις. Στην `(+ 2.3 1)` ο `int` (1) είναι στην κορυφή της στοίβας άρα αρκεί `i2f`. Στην `(+ 4 4.5)` ο `int` (4) είναι δεύτερος στη στοίβα, άρα απαιτείται `swar`, `i2f`, `swar` πριν το `fadd`.
- Αποτέλεσμα εκτέλεσης: επειδή σε κάθε πράξη υπάρχει πραγματικός τελεστής και τα δύο αποτελέσματα είναι `real`, το screenshot δείχνει 3.3 ($2.3+1$) και 8.5 ($4+4.5$) που είναι σωστά.

Παράδειγμα 4:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple4
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated -----
ST:: Name x of type 1 (i) in Position 2
ST:: Name y of type 2 (f) in Position 1

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
10.5
7
```

Σχολιασμός:

- Πηγαίος κώδικας: δηλώσεις (`float y`) και (`int x`), αναθέσεις (`= y 10.5`), (`= x (+ 3 4)`) και εκτυπώσεις. Δοκιμάζει την αποθήκευση/φόρτωση μεταβλητών.

- Αλλαγές/προσθήκες: προστέθηκαν οι κανόνες δήλωσης (int id)/(float id) με κλήση της addvar, ο κανόνας ανάθεσης (= id expr) με insertstore και η φόρτωση μεταβλητής στις εκφράσεις. Η y (float) παίρνει θέση 1, η x (int) θέση 2 όπως επιβεβαιώνει ο πίνακας συμβόλων στο screenshot.
- Αποτέλεσμα εκτέλεσης: το screenshot δείχνει τον Symbol Table (x type 1 (i) στη θέση 2, y type 2 (f) στη θέση 1) και την έξοδο 10.5 (η τιμή της y με fload/println(F)V) και 7 (η τιμή της x με iload/println(I)V).

Παράδειγμα 5:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple5
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated -----
ST:: Name x of type 1 (i) in Position 2
ST:: Name y of type 2 (f) in Position 1

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
14
```

Σχολιασμός:

- Πηγαίος κώδικας: (= y 10.5) (= x (+ 4 y)) όπου το y είναι float και το x int δοκιμάζει διπλή μετατροπή.
- Αλλαγές/προσθήκες: στην (+ 4 y) ο int (4) είναι δεύτερος στη στοίβα, άρα swap, i2f, swap και η πράξη γίνεται fadd (αποτέλεσμα real). Επειδή η μεταβλητή x είναι int αλλά η έκφραση real, στην ανάθεση γίνεται επιπλέον f2i πριν το istore, η δεύτερη μετατροπή που γίνεται στον κανόνα ανάθεσης.
- Αποτέλεσμα εκτέλεσης: ο πίνακας συμβόλων δείχνει x στη θέση 2 (int) και y στη θέση 1 (float), η έκφραση 4 + 10.5 = 14.5 περικρίπτεται σε int κατά την ανάθεση, οπότε η εκτέλεση τυπώνει 14, όπως φαίνεται στο screenshot.

Παράδειγμα 6:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple6
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated -----
ST:: Name z of type 1 (i) in Position 3
ST:: Name y of type 1 (i) in Position 2
ST:: Name x of type 1 (i) in Position 1

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
3
5
5
```

Σχολιασμός:

- Πηγαίος κώδικας: $(= x \ 3) (= y \ (x++))$ και $(= z \ (++x))$ εφαρμόζει τον επιθεματικό και τον προθεματικό τελεστή ++.
- Αλλαγές/προσθήκες: προστέθηκαν οι κανόνες T_id T_inc post-increment: πρώτα iload παλιάς τιμής, μετά iinc και T_inc T_id (pre-increment: πρώτα iinc, μετά iload νέας τιμής. Με αρχικό $x=3$: στο $x++$ η y παίρνει 3 και το x γίνεται 4. Στο $++x$ το x γίνεται 5 και η z παίρνει 5.
- Αποτέλεσμα εκτέλεσης: το screenshot δείχνει τον πίνακα συμβόλων με z,y,x (θέσεις 3,2,1) και τις τρεις εκτυπώσεις 3, 5, 5 (y, z και η τελική τιμή του x), που επιβεβαιώνουν τη σωστή σημασιολογία pre και post-increment.

Παράδειγμα 7:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple7
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated -----
ST:: Name y of type 1 (i) in Position 2
ST:: Name x of type 1 (i) in Position 1

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
19
```

Σχολιασμός:

- Πηγαίος κώδικας: $(= y \ 10) (= x \ + \ + \ 3 \ 5 \ (++y))$ με αρχικό $y=10$, ο τελεστής ++ μέσα σε φωλιασμένη προθεματική έκφραση.
- Αλλαγές/προσθήκες: η έκφραση $+ \ + \ 3 \ 5 \ (++y)$ ισοδυναμεί με $(3+5)+(++y)$. Εκπέμπεται iconst_3, iconst_5, iadd (=8) έπειτα το $(++y)$ ως iinc 2 1 (y από 10 σε 11) και iload 2 και τέλος iadd (8+11). Δείχνει ότι ο κανόνας του ++ ενσωματώνεται σωστά μέσα σε μεγαλύτερες εκφράσεις.
- Αποτέλεσμα εκτέλεσης: το screenshot δείχνει την έξοδο 19 $(= 3+5+11)$ που είναι το αναμενόμενο.

Παράδειγμα 8:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple8
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated -----
ST:: Name y of type 1 (i) in Position 2
ST:: Name x of type 1 (i) in Position 1

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
13
```

Σχολιασμός:

- Πηγαίος κώδικας: $(= x \ (< y \ 10 \ ? \ 13 \ : \ 23))$ με $y=8$ η έκφραση υπό συνθήκη.

- Αλλαγές/προσθήκες: προστέθηκε ο κανόνας ($\text{relop expr expr ? expr : expr}$) με τρεις ενέργειες και τρία labels. Φορτώνονται y και 10, εκπέμπεται if_icmplt #_1 (άλμα στο αληθές αν $y < 10$), αλλιώς goto #_2. Το true φορτώνει 13 και κάνει goto #3 παρακάμπτοντας τον ψευδή (23).
- Αποτέλεσμα εκτέλεσης: αφού $8 < 10$, επιλέγεται η τιμή 13. Το screenshot δείχνει την έξοδο 13, σύμφωνα με τη λογική των αλμάτων.

Παράδειγμα 9:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple9
** Cleaning up previous Files *****
Errors found 0.

Symbol Table Generated -----
ST:: Name y of type 1 (i) in Position 2
ST:: Name x of type 1 (i) in Position 1

** Compiler Finished! *****
** Starting Jasmin Assembler *****
** Assembler Finished! *****
** Executing code in Java *****
15
```

Σχολιασμός:

- Πηγαίος κώδικας: $(= x / + 10 (> y 0 ? 20 : 50) 2)$ με $y=10$, η υπό συνθήκη τιμή εμφωλευμένη μέσα σε αριθμητική έκφραση.
- Αλλαγές/προσθήκες: η έκφραση είναι $(10 + (y > 0 ? 20 : 50)) / 2$. Η υπό συνθήκη χρησιμοποιεί if_icmplt με τα ίδια τρία labels, επειδή $y=10 > 0$ επιστρέφει 20. Ακολουθεί iadd ($10+20=30$), iconst_2, idiv ($30/2=15$). Δείχνει ότι η τιμή υπό συνθήκη λειτουργεί ως κανονική υποέκφραση.
- Αποτέλεσμα εκτέλεσης: το screenshot δείχνει την έξοδο 15 ($(10+20)/2$), που είναι το σωστό αποτέλεσμα.

Παράδειγμα με error:

```
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples# ./compile_and_run simple_error
** Cleaning up previous Files *****
ERROR: syntax error, unexpected T_num, expecting print or int or float or = (line 2).
Variable y NOT declared, in code_line 4.ERROR: Variable Declation fault (line 4).
Variable z NOT declared, in code_line 5.ERROR: Variable Declation fault (line 5).
Variable w NOT declared, in code_line 6.ERROR: Variable Declation fault (line 6).
Errors found 4.
No Code Generated.

Symbol Table Generated -----
ST:: Name x of type 1 (i) in Position 1

** Compiler Finished! *****
No jasmin (.j) File Generated
user:/mnt/host/c/Users/owner/Desktop/coursework2_2025_ics23085/Exercise2/Examples#
```

Σχολιασμός:

- Πηγαίος κώδικας: το simple_error περιέχει σκόπιμα λάθη, μια συντακτικά λανθασμένη εντολή (1 2) και χρήση μη δηλωμένων μεταβλητών y, z, w.

- Αλλαγές/προσθήκες: για την ανίχνευση προστέθηκαν οι εντολές ERR_VAR_MISSING (μη δηλωμένη μεταβλητή) και ERR_VAR_DECL/ERR_TYPE_MISMATCH μαζί με τον κανόνα ανάκαμψης (error και τη συνάρτηση skip_to_stmt_end() που παραλείπει την υπόλοιπη εσφαλμένη εντολή ώστε να αναφερθούν πολλαπλά σφάλματα σε ένα πέρασμα.
- Αποτέλεσμα εκτέλεσης: όπως φαίνεται στο screenshot, αναφέρεται το συντακτικό σφάλμα στη γραμμή 2 και τα τρία σφάλματα μη δηλωμένης μεταβλητής στις γραμμές 4–6 (y, z, w). Συνολικά Errors found 4 και επειδή υπάρχουν σφάλματα παράγεται No Code Generated και No jasmin (.j) File Generated.
- Ο πίνακας συμβόλων περιέχει μόνο το x (η μόνη έγκυρη δήλωση (int x)) στη θέση 1, επιβεβαιώνοντας ότι οι υπόλοιπες μεταβλητές όντως δεν καταχωρήθηκαν.

Κώδικας:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern int code_line;
extern FILE *yyout;
int yylex();
int yyerror(const char *);
int no_errors = 0;

#define ERR_VAR_DECL(VAR,LINE) {fprintf(stderr,"Variable :: %s in code_line %d. ",VAR,LINE); yyerror("Var already defined");YERROR;}
#define ERR_VAR_MISSING(VAR,LINE) {fprintf(stderr,"Variable %s NOT declared, in code_line %d.",VAR,LINE); yyerror("Variable Declaration fault");YERROR;}
#define ERR_TYPE_MISMATCH(EXPECTED,LINE) {fprintf(stderr,"TYPE ERROR: Expected %s in code_line %d.",EXPECTED,LINE); yyerror("Type mismatch");YERROR;}

#include "jvmLangTypesFunctions.h"
#include "symbolTable.h"

ST_TABLE_TYPE symbolTable;
#include "codeFacilities.h"

/*forward declaration,defined in the epilogue after token names are available */
extern int g_paren_depth; /*maintained by the lexer on every ( and ) */
void skip_to_stmt_end(void);

/*
 * coerce_binary:emit type-conversion instructions for binary operations.
 * Stack layout (bottom -> top): [left_value] [right_value]
 * Returns the result type (type_real if either operand is real).
 *
 * Case 1:left=real,right=int + right (int) is on TOP -> just i2f
 * Case 2:left=int,right=real + left (int) is BELOW right -> swap,i2f,swap
 */
static PartType coerce_binary(PartType left, PartType right)
{
    if (left== type_integer && right== type_integer) return type_integer;
    if (left ==type_real && right==type_real) return type_real;
    if (left ==type_real && right== type_integer) {
        insertINSTRUCTION("i2f");
    } else {
        insertINSTRUCTION("swap");
        insertINSTRUCTION("i2f");
        insertINSTRUCTION("swap");
    }
    return type_real;
}

/* Labels for conditional expressions, shared across mid-rule actions */
static int cond_l1, cond_l2, cond_l3;

%}

#define parse.error verbose

%union{
    int intval;
    char *lexical;
    struct {
        PartType type;
        char *place;
    } se;
    RelationType relopIndex;
}

%token T_start "start"
%token T_end "end"
%token T_print "print"
%token T_int "int"
%token T_float "float"
%token T_inc "++"
%token T_plus "+"
%token T_minus "-"
%token T_mul "*"
}
```

```

%token T_plus "+"
%token T_minus "-"
%token T_mul "*"
%token T_div "/"
%token T_eq "=="
%token T_gt ">"
%token T_lt "<"
%token T_assign "="
%token T_qmark "?"
%token T_colon ":"
%token T_lparen "("
%token T_rparen ")"

%token <lexical> T_id
%token <lexical> T_num
%token <lexical> T_fnum

%type <se>      expr
%type <relopIndex> relop

%%

program:
    T_start T_id
    {
        create_preamble($2);
        symbolTable= NULL;
    }
    stmts T_end
    {
        insertINSTRUCTION("return");
        insertINSTRUCTION(".end method\n");
    }
    ;

stmts:
    /* empty */
    | stmts stmt
    ;

stmt:
    /*variable declaration: (int x) */
    T_lparen T_int T_id T_rparen
    {
        if (!addvar(&symbolTable, $3, type_integer))
            ERR_VAR_DECL($3, code_line);
    }

    /*variable declaration:(float y) */
    | T_lparen T_float T_id T_rparen
    {
        if (!addvar(&symbolTable, $3, type_real))
            ERR_VAR_DECL($3,code_line);
    }

    /*
     * Assignment: (= varname expr)
     * The declared variable check is a mid-rule action executed before
     * expr is parsed.This ensures that if the variable is undeclared,
     * YYERROR fires while the bison stack still has only T_lparen T_assign T_id,
     * so the error-recovery rule can call skip_to_stmt_end() and
     * skip all remaining tokens of this statement (including nested parens).
     */
    | T_lparen T_assign T_id
    {
        if (!lookup(symbolTable,$3))
            ERR_VAR_MISSING($3,code_line);
    }
    expr T_rparen
    {
        ParType vartype= lookup_type(symbolTable, $3);
        int varpos= lookup_position(symbolTable, $3);
        if (vartype== type_integer && $5.type == type_real)

```

```

        int varpos= lookup_position(symbolTable, $5);
        if (vartype== type_integer && $5.type == type_real)
            insertINSTRUCTION("f2i");
        else if (vartype== type_real && $5.type == type_integer)
            insertINSTRUCTION("i2f");
        insertSTORE(vartype, varpos);
    }

/* Print (print expr) */
| T_lparen T_print expr T_rparen
{
    insertINSTRUCTION("getstatic java/lang/System/out Ljava/io/PrintStream;");
    insertINSTRUCTION("swap");
    if ($3.type== type_integer)
        insertINVOKEVIRTUAL("java/io/PrintStream/println",type_integer,type_void);
    else
        insertINVOKEVIRTUAL("java/io/PrintStream/println",type_real, type_void);
}

/*
 * Error recovery: If a statement fails bison pops the stack until
 * it hits the 'error' token. At that point the T_lparen of the failing statement is on the stack.
 * skip_to_stmt_end() consumes the rest of that statement (handling
 * any nested parentheses) so we can resume parsing from the
 * next statement safely.
 */
| T_lparen error
{
    yyclearin; skip_to_stmt_end(); yyerrok;
}

;

relop:
    T_gt { $$= OP_GT; }
| T_lt { $$=OP_LT; }
| T_eq { $$ = OP_EQ; }
;

/*
 *expressions in prefix form
 */
expr:
    /*Integer literal */
    T_num
    {
        pushInteger(atoi($1));
        $$.type = type_integer;
    }

    /*Float literal */
| T_fnum
    {
        insertLDC($1);
        $$.type= type_real;
    }

    /* Variable load */
| T_id
    {
        if (!lookup(symbolTable, $1))
            ERR_VAR_MISSING($1, code_line);
        Partype vt= lookup_type(symbolTable, $1);
        int vp= lookup_position(symbolTable, $1);
        insertLOAD(vt, vp);
        $$.type =vt;
    }

    /* Post-increment:var++
     *load old value onto stack then increase the variable in memory */
| T_id T_inc
    {
        if (!lookup(symbolTable,$1))
            ERR_VAR_MISSING($1,code_line);
        if (lookup_type(symbolTable, $1)!= type_integer)
            ERR_TYPE_MISMATCH("int", code_line);
    }

```



```

        ERR_TYPE_MISMATCH("int", code_line);
        int vp= lookup_position(symbolTable, $1);
        insertLOAD(type_integer, vp);
        insertIINC(vp, 1);
        $$type= type_integer;
    }

    /*pre-increment:++var
    * Increase the variable in memory first then load the new value */
    | T_inc T_id
    {
        if (!lookup(symbolTable, $2))
            ERR_VAR_MISSING($2,code_line);
        if (lookup_type(symbolTable, $2)!= type_integer)
            ERR_TYPE_MISMATCH("int",code_line);
        int vp = lookup_position(symbolTable,$2);
        insertIINC(vp,1);
        insertLOAD(type_integer,vp);
        $$type=type_integer;
    }

    /*Addition:+ expr expr */
    | T_plus expr expr
    {
        ParType rt= coerce_binary($2.type, $3.type);
        insertOPERATION(rt, "add");
        $$type =rt;
    }

    /*subtraction:- expr expr */
    | T_minus expr expr
    {
        ParType rt= coerce_binary($2.type, $3.type);
        insertOPERATION(rt, "sub");
        $$type =rt;
    }

    /*multiplication:* expr expr */
    | T_mul expr expr
    {
        ParType rt= coerce_binary($2.type, $3.type);
        insertOPERATION(rt, "mul");
        $$type= rt;
    }

    /*division:/ expr expr */
    | T_div expr expr
    {
        ParType rt= coerce_binary($2.type, $3.type);
        insertOPERATION(rt, "div");
        $$type =rt;
    }

    /*
    * operator implementation (JVM bytecode)
    * Pattern: (relop lhs rhs ? true_val : false_val)
    *
    * Uses 3 labels (L1, L2, L3) for the jump logic.
    * Allocated at M1 in static globals (cond_l1/l2/l3).
    *
    * M1 (after relop lhs rhs):both operands are on the stack.
    *   -> Emit: if_icmp<relop> L1
    *   -> Emit: goto L2
    *   -> Emit: L1:
    *
    * M2 (after ? true_val):true_val is now on the stack.
    *   -> Emit: goto L3 (skip false branch)
    *   -> Emit: L2:
    *
    * Final (after : false_val ):false_val is on the stack.
    *   -> Emit: L3:
    */
    | T_lparen relop expr expr
    {
        /* M1 */
        {

```



```

T_rparen expr T_colon expr T_rparen
/* M1 */
{
    cond_l1= Label();
    cond_l2 =Label();
    cond_l3= Label();
    insertCMPPOP($2,cond_l1);
    insertGOTO(cond_l2);
    insertLabel(cond_l1);
}
T_qmark expr
/* M2 */
{
    insertGOTO(cond_l3);
    insertLabel(cond_l2);
}
T_colon expr T_rparen
{
    insertLabel(cond_l3);
    $$.$type= $7.$type;
}

/* Parenthesised expression: ( expr ) */
T_lparen expr T_rparen
{
    $$= $2;
}

;

%%

/*
*skip_to_stmt_end:skip all tokens until the closing ')' of the current statement is consumed.The opening '(' has already been shifted by
*bison,so we start at depth=1 and stop when depth reaches 0.This safely handles deeply nested parentheses.*/
void skip_to_stmt_end(void)
{
    /* g_paren_depth reflects how many '(' are still open,
    counting the outer statement paren plus any nested ones that
    were already consumed before the error occurred.We simply
    keep reading tokens (the lexer updates g_paren_depth itself)
    until we are back to the depth that existed before this
    statement's opening '(' was read for example depth 0. */
    int tok;
    while (g_paren_depth>0){
        tok =yylex();
        if (tok== 0) break; /* EOF safety */
    }
}

int yyerror(const char * msg)
{
    fprintf(stderr,"ERROR: %s (line %d).\n", msg, code_line);
    no_errors++;
    return 0;
}

#include "jvmLExp.lex.c"

int main(int argc,char **argv){
    FILE* jasmin_file;
    ++argv,--argc;
    if ( argc>0 && (yyin = fopen( argv[0], "r"))== NULL)
    {
        fprintf(stderr,"File %s NOT FOUND in current directory.\n Using stdin.\n",argv[0]);
        yyin = stdin;
    }
    int result= yyparse();
    fprintf(stderr,"Errors found %d.\n",no_errors);
    if (no_errors == 0){
        if (argc > 1) {jasmin_file = fopen(argv[1], "w");}
        else {
            fprintf(stderr,"No second argument defined. Output to screen.\n\n");
            jasmin_file = stdout;
        }
        print_int_code(jasmin_file);
        fclose(jasmin_file);
    }
    print_int_code(jasmin_file);
    fclose(jasmin_file);
}

if (no_errors != 0)
    fprintf(stderr,"No Code Generated.\n");
print_symbol_table(symbolTable);
return result;
}

```